

Independent Adversarial Review and Remediation

T115 Capstone · Feature-003 · Product Backlog US-18 — Structured Advisor Briefing
Workflow Testing

Document date: 17 September 2026 · **Repository:** karen-lee1029/Capstone_T115 · **Branch:** feat/feature-003-implementation

This document records an independent adversarial review of the Feature-003 verification suite, conducted by a separate agent with no access to the implementing agent's reasoning, and the remediation carried out in response to its six findings.

It is organised in five parts: the review method and the prompt issued to the reviewer; the review findings verbatim; the remediation applied to each finding; the evidence that each finding is closed; and the resulting state of the feature.

1. Review method

1.1 Purpose

Feature-003 is a verification suite. It adds no product capability; it tests a briefing workflow delivered by Feature-001 (US-08) and Feature-002 (US-15). Because the implementing agent both wrote the tests and assessed whether they were adequate, an independent check was commissioned specifically to attack that self-assessment.

The reviewer was a separate model session — **Codex Astra-6 High** — with memory disabled, no access to the implementation conversation, and no pasted discussion. It received only artifacts: requirements, changed files, raw command output, and a runnable copy of the project.

1.2 Context independence

The review package was constructed so that the reviewer would form its own conclusions before encountering any claim made by the implementing team. Materials were sequenced into a first pass (requirements, changed code, runnable snapshot, raw evidence) and a read-second folder holding the plan, task list, rationale documents and the unfiltered patches, all of which contain self-assessment: a Constitution Check marked "PASS", thirty-four task checkboxes ticked as complete, and written justifications for each design decision.

Two iterations of package review were carried out before the package was issued. The first identified missing dependency files, absent pre-existing tests and the absence of separate per-change patches. The second identified that the unfiltered patches still carried the planning documents inside them, defeating the sequencing. Both were corrected before the review began.

1.3 The prompt issued to the reviewer

The following prompt was issued verbatim to Codex Astra-6 High:

*Conduct an independent adversarial review. The project is T115 Feature 003, all relevant files are found within the associated folder provided to Codex found at this path
C:\Users\renny\Desktop\Feature-003-Review-Package. Use the readme file specifically created for*

you as the independent reviewer. Files sourced for this prompt were created by extracting all Feature 003 related files generated and committed to the GitHub Repository for independent adversarial review.

The referenced readme (00-START-HERE.md) carried the standing instruction: *"Review independently. Do not assume the implementation is correct. Do not infer intent beyond the supplied artifacts."* It stated the constraints the work claimed to satisfy, but deliberately offered no suggested lines of attack, so that the reviewer's angles of enquiry would be its own.

1.4 Method used by the reviewer

The reviewer ran the suite in a temporary copy using the pinned dependency set under Python 3.11.15, then applied **fault injection**: one temporary change at a time, each restored afterwards. This is the decisive technique. A suite that passes proves only that it passes; a suite that continues to pass while a fault is present proves it is not watching.

Three injected faults survived the complete suite of 143 tests. Each corresponds to a finding below.

2. Review findings, verbatim

The reviewer's report is reproduced here without alteration. Its recommendation was **request changes**, with six material findings, all rated P2 — required corrections before claiming specification completion.

2.1 Summary as delivered

*Reviewed 17 September 2026. **Recommendation: request changes.** Six material findings remain in the verification suite and its traceability deliverable. All are P2: required corrections before claiming specification completion. This review did not identify a confirmed regression introduced by the separate blank-content production fix.*

*The unmodified package passes **143 tests** and Ruff. That result does not establish the claimed completeness: three independent fault injections each left all 143 tests passing. They introduced fabricated retry criteria, briefing text in exception tracebacks, and a REST storage-outage response incorrectly reported as absence.*

2.2 Finding 1 — The privacy check ignores exception tracebacks

Requirements cited: FR-034, SC-011; constitution X.

The helper joins only `record.getMessage()`. That excludes exception information rendered by a logging formatter. Consequently, the test can approve a log record whose visible traceback contains the exact briefing-text sentinel it is supposed to exclude.

*Reproduction: I inserted `logger.exception("generation failed")` into the existing first-attempt generation-exception handler in the disposable copy. **All 143 tests passed.** The failure-path fixtures already put `SECRET_TEXT` into the exception message.*

Correction: check rendered log output including exception information, and inspect structured fields where those are part of operational logging.

2.3 Finding 2 — The feedback tests do not establish "exactly as reported, nothing added"

Requirements cited: FR-024, SC-010; User Story 2 acceptance scenario 3.

The direct test checks three substrings. It cannot reject additional fabricated criteria or feedback. The supposedly stronger provider/validator pairing responds only to the fixed header `The previous attempt did not pass validation.`, rather than the actual reported criteria and feedback.

*Appending - `PLACEHOLDER_CRITERION_INVENTED_BY_RETRY` to the retry criteria block left **all 143 tests passing**. Replacing the appended content with just the header, removing all criteria and feedback, still passed the test claiming causal assurance.*

Correction: assert exact preservation of the supplied payload and reject additional payload values. Exercise criteria-only and feedback-only inputs as well as both together and neither. If retained, the conditional double must depend on the actual reported synthetic values.

2.4 Finding 3 — Storage-read failures are missing from cross-boundary coverage

Requirements cited: FR-032, FR-033, SC-015; edge case "Governed storage is unreachable while reading".

The existing REST retrieval test covers a stored briefing, absence and an unknown student. Its storage-error test covers a POST write failure. Neither exercises a GET read failure.

*Reproduction: I changed the REST retrieval exception response from 503 / Validated briefing store unavailable to 404 / No validated briefing available. **All 143 tests passed.** This is precisely the outage-versus-absence distinction the specification requires.*

The statement that every REST outcome is already covered, repeated in research R8 and T022, is false.

2.5 Finding 4 — Governed-storage parity omits the required failed-write outcome

Requirements cited: FR-031; User Story 3 acceptance scenarios 5 and 7.

None sets `FakeFilesClient.fail_upload` or otherwise causes a write failure while the workflow uses `VolumeBriefingStore`. The number "six" has been preserved, but the six required scenario categories have not: the success category is split into two entries, while the failed-write category is absent.

2.6 Finding 5 — Several new tests duplicate behaviour already adequately verified

Requirements cited: FR-005, SC-004; constitution XII.

Three concrete overlaps were identified, each a new verification asserting the same observable property as an existing one.

These are assertion-level overlaps, not objections to reusing a scenario to observe a previously untested property. The genuinely new shared ordering assertions are useful. In contrast, relabelling an existing call-count assertion as a "matrix-wide" property does not add assurance.

2.7 Finding 6 — The traceability record is incomplete and leaves dependencies outside its self-check

Requirements cited: FR-039, FR-040, SC-012, SC-016.

*The scenario map contains **25 of the 34 new test functions**. Nine are absent. Tracked re-verification also names one retry test and refers to "the eight sibling scenarios," and similarly uses unnamed siblings for storage. The regex resolves explicit names; it cannot guard those unnamed dependencies.*

Reproduction: renaming the old

*`test_configuration_error_on_retry_generation_is_re_raised_not_terminal` so it was no longer a test reduced collection to **142 passing tests**, with the traceability check still passing.*

2.8 What the review confirmed as sound

The reviewer independently reproduced the reported baseline of 143 passing tests and a clean Ruff run, confirmed the filtered patches exactly reproduce the supplied files, confirmed no production-source edits in the implementation change and no changes to the thirteen pre-existing test files, and confirmed the suite makes no external network access under an auditing hook. It found no defect in the separate blank-content production fix.

3. Remediation

Each finding was addressed within the feature's testing-only scope. No production source file was modified. Fault injections used to verify the fixes were applied to disposable copies and restored.

3.1 Finding 1 — rendered log output including tracebacks

A new helper renders each captured record through a logging formatter, so exception information appears in the text the privacy assertion inspects, and additionally appends any non-standard structured fields, since structured handlers emit those.

A verification was added for the check itself. It raises an exception carrying the briefing sentinel, logs it with `logger.exception`, then asserts three things: that the message alone does not contain the sentinel, that the rendered output does, and that the privacy assertion rejects the rendered output. Without this, a future regression in the helper would silently weaken every privacy scenario that depends on it.

3.2 Finding 2 — exact payload relay

Three changes were made.

The substring assertions were replaced with **set equality** over recognisable payload values. The scenario compares the set of synthetic values appearing in the text the retry appended against the set validation actually reported. A fabricated addition changes the set and fails; a substring check cannot detect it.

The conditional generation double now keys on the **reported values themselves** rather than the wrapper header. It emits revised text only when every reported value is present in the prompt it received. A pass therefore depends on those specific values arriving, which is what the causal claim asserted and the header-based version did not deliver.

The cases were parametrised across criteria-and-feedback, criteria-only and feedback-only, with the neither case retained separately. A converse scenario was added: when the reported values do not arrive, the second attempt must not pass. Without it, the propagation scenario could pass for unrelated reasons.

The assertions deliberately target payload values rather than wrapper wording, since the wrapper belongs to US-12 and must not become an acceptance criterion here.

3.3 Finding 3 — read-outage coverage across boundaries

Seven scenarios were added, injecting list and download failures through the existing in-memory files client against the real governed store.

At the **service** boundary an outage raises an explicit storage error while genuine absence returns nothing. At the **REST** boundary an outage is 503 "store unavailable" while absence is 404 "none available", and the two statuses are asserted to differ. At the **tool** boundary an outage raises rather than answering "not available". A further scenario confirms that a not-found from the files client means absence, not failure, so the distinction holds in both directions.

The traceability record's REST claim was corrected: Feature-001 covers the briefing **write** outcomes at that boundary, not the read outage.

3.4 Finding 4 — governed failed-write parity

A scenario was added that seeds a prior briefing in the governed store, drives a valid briefing through the workflow, fails the upload at the files client, asserts the explicit storage failure, then re-enables reads and confirms the prior briefing survived. This exercises the path the other parity entries missed: the in-memory failing store raises before reaching a real store, and Feature-002's upload-failure verification exercises the

adapter in isolation rather than the workflow.

3.5 Finding 5 — duplicate assertions removed

Three verifications were removed from the Feature-003 suite. Each asserted an observable property an existing verification already asserted. They are now cited individually in the traceability record's tracked re-verification section rather than re-implemented.

One verification was retained but narrowed. Its call-count assertion duplicated an existing one; its sequence assertion — that validation never appears on a path where no draft was produced — did not. It now asserts the sequence.

3.6 Finding 6 — complete mapping and bidirectional self-check

All forty Feature-003 verifications are mapped in the scenario map, grouped by file. Sibling shorthand was replaced throughout with explicit file-and-name citations, adding roughly thirty named references across the retry, store-contract and boundary sections.

A second self-check was added. The existing one guards *cited implies exists*; on its own that permits a scenario to be added, or an existing one renamed, without the record noticing — which is exactly what the reviewer demonstrated. The new check guards *exists implies mapped*, so the record's completeness claim is verified rather than asserted.

4. Verification that each finding is closed

Each fault the reviewer injected was re-applied to the remediated suite, then restored. Every one now fails the suite.

Injected fault	Before remediation	After remediation
<code>logger.exception</code> leaking briefing text via traceback	143 passed — undetected	Fails privacy scenario
Fabricated criterion appended to the retry block	143 passed — undetected	Fails exact-relay scenario, two cases
REST read outage reported as 404 absence	143 passed — undetected	Fails boundary scenario, two cases
Cited verification renamed out of existence	Check still passed	Fails cited-resolves check
Unmapped scenario added to the suite	No check existed	Fails new completeness check

The fourth row is the reviewer's own drift demonstration. It previously escaped because the renamed verification was covered by sibling shorthand rather than named; it is now cited explicitly and therefore guarded.

The fifth row is a fault the reviewer did not need to inject, because no check existed to defeat. It is included to demonstrate the new completeness check does what it claims.

4.1 Resulting suite

```
before remediation: 143 passed (3 injected faults survived)
after remediation:  153 passed (every injected fault detected)

ruff check .          All checks passed
```

Verification file	Before	After
<code>test_briefing_workflow_storeage.py</code>	18	19
<code>test_briefing_workflow_boundaries.py</code>	7	16
<code>test_briefing_workflow_outcomes.py</code>	13	13
<code>test_briefing_workflow_traceability.py</code>	10	11
<code>test_briefing_workflow_retry.py</code>	11	10
Pre-existing Feature-001 and Feature-002 suites	84	84

The retry file has one fewer verification despite gaining three: three duplicative ones were removed under Finding 5. This is the intended direction — the constitution's proportionate-testing principle treats redundant tests as a defect, not a surplus.

4.2 Scope confirmation

The remediation held the same constraints as the original work.

Constraint	Evidence
No production source file modified	<code>git status</code> on <code>src/</code> clean after every fault injection was restored
No pre-existing test file modified	None of the thirteen files appears as modified
Offline by construction	No network, workspace or credential use; the reviewer independently confirmed zero external access attempts
No new dependency, plugin or fixture framework	The support module still imports the three Feature-002 doubles unchanged

Files changed by the remediation: five Feature-003 verification files and the traceability record.

5. Assessment

5.1 What the review established

The central lesson is methodological. The original suite passed 143 tests and a clean lint run, and those facts were reported as evidence of completeness. They were not. Three separate faults — one of them a privacy leak of exactly the kind the suite was written to prevent — sat undetected behind a green result.

Fault injection, not test count, is what distinguishes a suite that watches from a suite that merely runs. The reviewer's decision to mutate rather than read is why these findings surfaced at all.

A second lesson concerns self-checking records. The traceability record was designed to be self-verifying, and that design was sound in one direction only. Guarding *cited implies exists* while leaving *exists implies mapped* unguarded produced a record that could report completeness it did not have. A one-directional check on a two-directional claim is a check that will eventually mislead.

5.2 Limits that remain

These are inherent to the feature's scope and are not defects.

The suite drives generation and validation with controlled outcomes, because the concrete implementations are later backlog stories. It can establish orchestration behaviour conditional on those outcomes. It cannot establish actual model responsiveness, briefing quality, final acceptance criteria, live governed-storage behaviour, or successful integration of the future US-12, US-13 and US-14 implementations.

Feature-001 SC-007 remains unverified, recorded in the traceability record with its reason: the US-12 and US-14 work is expected to be delivered outside the seam boundary, so the substitution that criterion describes is not anticipated.

The exact-relay assertion recognises fabricated values drawn from the synthetic vocabulary the fixtures use. An arbitrary fabricated string outside that vocabulary would not be detected by set comparison alone. This is a bounded limit, stated rather than hidden.

5.3 Outstanding

The reviewer noted that planning claims should be revised once the findings are resolved — specifically the statements in the plan and rationale documents asserting that every REST outcome was already covered, which Finding 3 disproved, and the task-completion assessments for the affected tasks. That correction is

pending and is the remaining work arising from this review.